Nonstationarity-Aware Mamba for Financial Time Series Forecasting

Draft Manuscript

April 26, 2026

Abstract

Financial time series forecasting is difficult because the data-generating process can change across time. Changes in level, volatility, trend, and temporal regime can make a model trained on past windows unreliable for future horizons. This manuscript studies nonstationarity-aware forecasting with Mamba-based sequence models, using daily GLD close-price prediction as the initial testbed. The proposed model, NS-Mamba, extends an S_Mamba backbone with adaptive normalization, series decomposition, and distribution-shift-aware fusion. Before evaluating the proposed architecture, we define a controlled baseline protocol across 12 horizons from 1 to 360 days and lock a shared configuration through S_Mamba calibration. This draft provides the full manuscript structure, problem formulation, model design, and experiment protocol. Final quantitative claims will be added only after the full GLD baseline table and NS-Mamba ablations are complete.

1 Introduction

Time series forecasting models are often evaluated under the assumption that past and future windows are drawn from similar distributions. This assumption is weak for financial data. Gold-related exchange-traded products such as GLD can experience abrupt level shifts, volatility changes, macroeconomic regime changes, and long periods of trend persistence or reversal. These effects are not just noise; they change the forecasting problem itself.

Recent deep forecasting models use attention, decomposition, and state-space mechanisms to model long temporal context. Mamba is especially attractive because it provides linear-time sequence modeling through selective state spaces [1]. In a forecasting setting, however, using Mamba alone does not guarantee robustness to nonstationarity. The model may still entangle level, scale, trend, and residual dynamics in one hidden representation. This motivates an architecture that keeps the efficient Mamba backbone while explicitly exposing nonstationary structure to the network.

This work begins with a strict experimental principle. Before introducing NS-Mamba, all suitable baselines are evaluated under a shared configuration selected by S_Mamba calibration. This prevents model-specific tuning from becoming a hidden source of performance differences. The same horizon suite and output scale are then used for clean baselines, Mamba variants, and the proposed NS-Mamba models.

The intended contributions are:

- a controlled GLD forecasting protocol covering horizons from 1 to 360 days;
- a clean comparison between Transformer-style, decomposition-based, inverted, and Mamba-based forecasting models;

- a nonstationarity-aware Mamba architecture combining adaptive normalization, decomposition, and shift-aware fusion;
- an ablation plan that isolates the contribution of each proposed module.

2 Related Work

2.1 Long-Horizon Time Series Forecasting

Long-horizon time series forecasting aims to predict many future steps from a historical lookback window. It is important in domains where decisions must be made before the target is observed, including finance, energy, traffic, weather, and operations planning. Compared with short-term forecasting, long-horizon forecasting is more sensitive to error accumulation, distribution shift, trend changes, and the difficulty of preserving useful long-range temporal information. In financial forecasting, these challenges are amplified because the target can move under changing macroeconomic conditions, market expectations, and volatility regimes. The GLD task studied in this manuscript follows this setting by evaluating horizons from 1 day to 360 days.

2.2 Transformer-Based Time Series Forecasting

Transformer-based forecasting models introduced attention mechanisms for modeling long temporal context. Informer improves efficiency by reducing the cost of attention for long sequences [10]. Reformer reduces attention complexity through locality-sensitive hashing and reversible layers [12]. Autoformer introduces decomposition into the forecasting architecture, separating trend and seasonal components to improve long-term prediction [11]. These models provide important baselines because they represent different design choices for efficient long-context modeling, decomposition, and temporal dependency extraction.

2.3 Inverted Forecasting Architectures

iTransformer argues that treating variables as tokens and encoding their temporal histories can be effective for multivariate forecasting [9]. The S_Mamba baseline in this repository follows a related inverted-token view by embedding each variable's lookback history before applying Mamba encoder layers. For the initial GLD setting, the task is single-target forecasting, but the same design remains relevant because it controls how temporal history is mapped into hidden features.

2.4 State Space Models and Mamba

State space models provide an alternative to attention for sequence modeling. The continuous-time form of a linear state space model is:

$$\mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}\mathbf{x}(t), \qquad \mathbf{y}(t) = \mathbf{C}\mathbf{h}(t), \tag{1}$$

where $\mathbf{h}(t)$ is the latent state, $\mathbf{x}(t)$ is the input, and $\mathbf{y}(t)$ is the output. After discretization with step size Δ, the recurrence can be written as:

$$\mathbf{h}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}\mathbf{x}_t, \qquad \mathbf{y}_t = \mathbf{C}\mathbf{h}_t, \tag{2}$$

with

$$\bar{\mathbf{A}} = \exp(\Delta\mathbf{A}), \qquad \bar{\mathbf{B}} = (\Delta\mathbf{A})^{-1}\left(\exp(\Delta\mathbf{A}) - \mathbf{I}\right)\Delta\mathbf{B}. \tag{3}$$

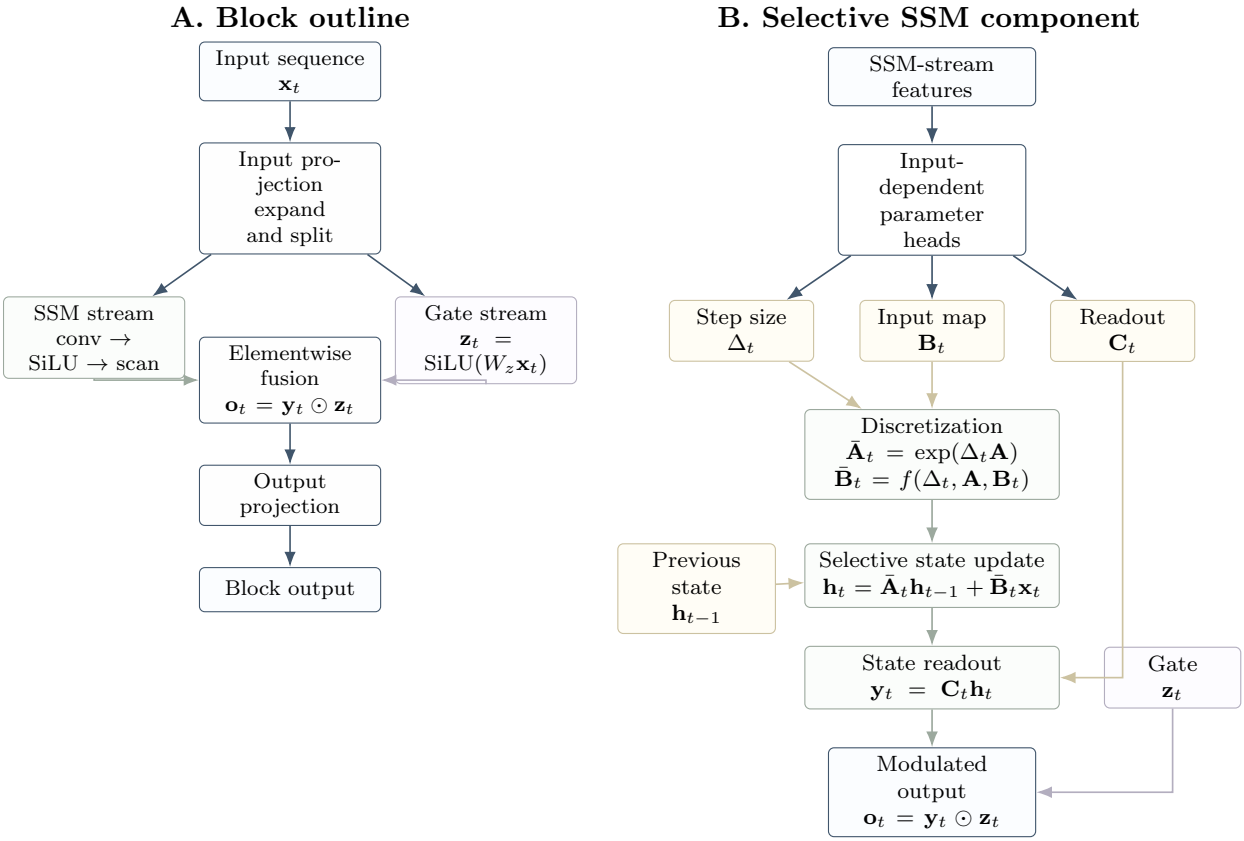


Figure 1: Mamba block architecture drawn with native TikZ so mathematical notation is rendered by LaTeX. Panel A shows the high-level block outline: input projection, SSM stream, gating stream, fusion, and output projection. Panel B expands the key selective state-space component, where input-dependent parameters generate the discretized update, readout, and gated output.

Classical structured state space models use fixed transition parameters after training. Mamba introduces selective state spaces, where Δ_t, $\mathbf{B}_t$, and $\mathbf{C}_t$ depend on the current input token [1]. The input-dependent recurrence is:

$$\mathbf{h}_t = \bar{\mathbf{A}}_t\mathbf{h}_{t-1} + \bar{\mathbf{B}}_t\mathbf{x}_t, \qquad \mathbf{y}_t = \mathbf{C}_t\mathbf{h}_t, \tag{4}$$

where $\bar{\mathbf{A}}_t$ and $\bar{\mathbf{B}}_t$ are computed from the input-dependent step size and input projection. This selectivity is useful for forecasting because the model can adapt what information is retained across time. Figure 1 summarizes the Mamba block used as the conceptual backbone in this manuscript. The central question is whether explicit nonstationarity modules can make this selective mechanism more effective for data whose distribution changes across windows.

2.5 Mamba-Based Time Series Forecasting

Several recent studies have adapted Mamba to time series forecasting. S_Mamba studies whether Mamba is effective for time series forecasting and uses a bidirectional Mamba layer to model inter-variate correlations with a feed-forward layer for temporal dependencies [2]. Bi-Mamba+ extends this direction with forward and backward Mamba processing, a forget gate, and a relation-aware choice between channel-independent and channel-mixing tokenization [3]. MambaTS analyzes limitations of directly applying Mamba to long-term forecasting and proposes temporal Mamba blocks,

variable scanning strategies, and variable-aware scan ordering [4]. TimeMachine uses multiple Mamba pathways to handle channel-mixing and channel-independence settings while preserving linear scalability for long-term forecasting [5]. TSMamba moves from task-specific forecasting toward a Mamba-based foundation model, using forward and backward Mamba encoders and transfer learning for time series prediction [6]. Together, these examples show that Mamba has become an active backbone for forecasting, but they do not directly resolve the specific problem studied here: explicit nonstationarity handling through adaptive normalization, decomposition, and shift-aware fusion.

2.6 Nonstationarity and Normalization

Distribution shift is a central issue in time series forecasting. RevIN uses reversible instance normalization to reduce distribution shift between training and testing windows while restoring the original scale for prediction [7]. Non-stationary Transformers introduce mechanisms to handle stationarity and nonstationarity within Transformer forecasting [8]. NS-Mamba follows the same broad motivation but studies it in a Mamba-based architecture with adaptive normalization, decomposition, and shift-aware fusion.

3 Problem Formulation and Nonstationarity

Let

$$\mathbf{X}_{t-L+1:t} \in \mathbb{R}^{L\times C} \tag{5}$$

denote an input window of length L with C variables. The goal is to predict

$$\mathbf{Y}_{t+1:t+H} \in \mathbb{R}^{H\times C_o}, \tag{6}$$

where H is the forecast horizon and C_o is the output dimension. For the GLD study, $C = C_o = 1$ and the target is the daily `close` column.

The evaluated horizon set is:

$$H \in \{1, 3, 5, 7, 14, 30, 60, 90, 120, 180, 270, 360\}. \tag{7}$$

These horizons cover short-term, middle-term, and long-term forecasting behavior.

Nonstationarity means that the joint distribution of input and output windows changes with time. Let $\mathcal{D}_t$ denote the local joint distribution around time t. The nonstationary setting can be written as:

$$\mathcal{D}_t(\mathbf{X}_{t-L+1:t}, \mathbf{Y}_{t+1:t+H}) \neq \mathcal{D}_{t+\tau}(\mathbf{X}_{t+\tau-L+1:t+\tau}, \mathbf{Y}_{t+\tau+1:t+\tau+H}). \tag{8}$$

This mismatch means that minimizing empirical training risk does not necessarily minimize future test risk. In GLD forecasting, common sources of mismatch include changing price level, volatility expansion or contraction, trend reversal, and regime-specific temporal dependence.

The proposed approach is based on three assumptions. First, local level and scale variation should be normalized before sequence modeling. Second, trend-like and residual-like movements should not always be forced into the same representation. Third, the model should receive explicit shift statistics so that it can change its fusion behavior under different regimes.

4 Method: NS-Mamba

NS-Mamba is designed as a modular extension of S_Mamba. The base model provides the sequence backbone, while the new modules expose nonstationary structure. Given an input window $\mathbf{X}$, the model computes:

$$\hat{\mathbf{Y}} = \mathrm{DeNorm}\left(F_\theta\left(\mathrm{Decomp}\left(\mathrm{AdaptiveNorm}(\mathbf{X})\right), \phi(\mathbf{X})\right)\right), \tag{9}$$

where F_θ is the Mamba forecasting network and $\phi(\mathbf{X})$ is a vector of distribution-shift statistics.

4.1 Adaptive Normalization

The first module normalizes the input window and learns adaptive affine parameters from window statistics. The normalized input is:

$$\tilde{\mathbf{X}} = \gamma(\phi(\mathbf{X})) \odot \frac{\mathbf{X} - \mu(\mathbf{X})}{\sigma(\mathbf{X}) + \epsilon} + \beta(\phi(\mathbf{X})). \tag{10}$$

Here $\mu(\mathbf{X})$ and $\sigma(\mathbf{X})$ are local window statistics, while $\gamma(\cdot)$ and $\beta(\cdot)$ are learned functions. This module generalizes fixed reversible normalization by allowing the normalized representation to depend on distribution-shift features.

4.2 Series Decomposition

The second module decomposes the normalized window into trend, seasonal, and residual components:

$$\tilde{\mathbf{X}} = \tilde{\mathbf{X}}^{\mathrm{trend}} + \tilde{\mathbf{X}}^{\mathrm{seasonal}} + \tilde{\mathbf{X}}^{\mathrm{residual}}. \tag{11}$$

The initial implementation uses moving-average decomposition. The purpose is not to assert a perfect additive data-generating process, but to provide an architectural bias that separates slow and fast temporal variation.

4.3 Shift-Aware Fusion

The third module uses the shift statistics $\phi(\mathbf{X})$ to control the fusion of hidden representations. For branch outputs $\mathbf{z}_k$, the fusion weights are:

$$\boldsymbol{\alpha} = \mathrm{softmax}\left(\mathbf{W}_\alpha \phi(\mathbf{X}) + \mathbf{b}_\alpha\right), \tag{12}$$

and the fused representation is:

$$\mathbf{z} = \sum_k \alpha_k \mathbf{z}_k. \tag{13}$$

This design allows the model to emphasize trend, seasonal, or residual branches differently when the input window exhibits different shift patterns.

5 Neural Network Architecture

5.1 Input and Tensor Shapes

The implementation uses tensors of shape $\mathbb{R}^{B \times L \times C}$, where B is batch size. For GLD, $C = 1$. After adaptive normalization and optional decomposition, each component is passed to an inverted embedding layer. The embedding maps temporal history into hidden dimension d_{model}. In the locked GLD setting, $d_{\mathrm{model}} = 64$ and the feed-forward dimension is 128.

5.2 S_Mamba Encoder

The S_Mamba backbone applies bidirectional Mamba encoder layers to the embedded representation. A simplified state-space update is:

$$\mathbf{h}_t = \mathbf{A}(\mathbf{x}_t)\mathbf{h}_{t-1} + \mathbf{B}(\mathbf{x}_t)\mathbf{x}_t, \qquad \mathbf{o}_t = \mathbf{C}(\mathbf{x}_t)\mathbf{h}_t. \tag{14}$$

The input-dependent matrices give Mamba a selective memory mechanism. NS-Mamba preserves this mechanism and adds explicit nonstationary conditioning before and around the encoder.

5.3 Branch Design

NS-Mamba-A contains the adaptive normalization block and one S_Mamba forecasting pathway. NS-Mamba-B adds decomposition and evaluates whether branch-specific modeling improves middle- and long-horizon forecasts. NS-Mamba-C adds shift-aware fusion. The branch encoders can be implemented with shared or component-specific parameters; the manuscript will report the final implementation choice once the model is fixed in code.

5.4 Forecast Head and Loss

The forecast head projects the final hidden representation to H future values. The training objective is mean squared error over the forecast horizon:

$$\mathcal{L} = \frac{1}{BHC_o} \sum_{b=1}^{B} \sum_{h=1}^{H} \sum_{c=1}^{C_o} \left(\hat{Y}_{b,h,c} - Y_{b,h,c} \right)^2. \tag{15}$$

In the current codebase, inverse transformation is enabled by default, so saved predictions, targets, and metrics are reported on the original GLD price scale unless a run uses `--no-inverse`.

6 Experimental Setup

6.1 Dataset

The initial dataset is GLD daily data stored at `data/GLD/GLD.csv`. The target column is `close`. The first experimental phase uses single-target forecasting. Future extensions may add more financial or macroeconomic variables, but those are outside the current baseline protocol.

6.2 Data Splits

The repository uses the standard custom-data split from the forecasting codebase. The validation and test windows use historical lookback context from the previous split to construct the input sequence. This is a common forecasting implementation detail, but it should be reported clearly because the input context for validation and test includes observations before the split boundary.

6.3 Baseline Models

The core baseline set is:

- S_Mamba;
- Transformer;

- Informer;
- Reformer;
- Autoformer;
- iTransformer.

Optional Mamba-augmented variants will be reported separately if they are included. The clean baselines should not be tuned independently after the shared setting is locked.

6.4 Shared Configuration

S_Mamba calibration selected Setting A as the locked baseline configuration:

Parameter	Value
Lookback length	360
Label length	180
Prediction length	horizon-specific
Model dimension	64
Feed-forward dimension	128
Encoder layers	2
Batch size	16
Target	`close`
Output scale	original price scale

All suitable models will use this configuration. If a model cannot run under this setting, the failure will be recorded rather than changing only that model's hyperparameters.

6.5 Metrics and Logging

The primary metrics are MSE and MAE. RMSE, MAPE, and MSPE are also saved when available. Training, validation, and test metrics are logged to TensorBoard. Final arrays are saved as `pred.npy`, `true.npy`, and `metrics.npy`, with run metadata stored in `meta.json`.

7 Results

The full result table is not complete yet. This section will report GLD baseline and NS-Mamba results after all runs finish under the locked configuration. The planned table format is:

Model	Horizon	MSE	MAE	Status
S_Mamba	1	pending	pending	pending
S_Mamba	360	pending	pending	pending
Transformer	1	pending	pending	pending
Transformer	360	pending	pending	pending

The final version of this section should include:

- one full table across all 12 horizons and all baseline models;

- a short-horizon summary for 1, 3, 5, and 7 days;
- a middle-horizon summary for 14, 30, 60, 90, and 120 days;
- a long-horizon summary for 180, 270, and 360 days;
- an error-curve visualization comparing representative horizons.

8 Ablation Study

The ablation study will isolate the contribution of each NS-Mamba component. The planned comparison is:

Model	Adaptive Norm	Decomposition	Shift Gate
S_Mamba	No	No	No
NS-Mamba-A	Yes	No	No
NS-Mamba-B	Yes	Yes	No
NS-Mamba-C	Yes	Yes	Yes

The ablation should be interpreted by horizon group. Adaptive normalization is expected to matter when level and scale shifts dominate. Decomposition is expected to matter more at middle and long horizons. Shift-aware fusion is expected to help when the input window shows strong recent-vs-full-window distribution changes. These are hypotheses for the experiment, not completed findings.

9 Discussion and Limitations

This manuscript is currently a draft before final experiments. The main limitation is that the proposed NS-Mamba variants have not yet been evaluated against the complete GLD baseline table. The results section therefore avoids performance claims.

The second limitation is dataset scope. GLD provides a useful nonstationary financial testbed, but one dataset is not enough to establish general forecasting superiority. After GLD experiments are complete, the method should also be tested on standard long-horizon forecasting datasets such as Exchange, Weather, ETT, Electricity, and Traffic.

The third limitation is split sensitivity. Financial time series evaluation can change substantially with different historical periods. The final manuscript should report the exact date range, train-validation-test boundaries, random seeds, and whether validation or test windows use lookback context from earlier splits.

Finally, the architecture may add complexity. The ablation study is necessary to determine whether each module improves forecasting enough to justify its additional parameters and implementation cost.

10 Conclusion

This manuscript presents a structured plan for nonstationarity-aware Mamba forecasting. The proposed NS-Mamba architecture combines adaptive normalization, decomposition, and shift-aware fusion around an S_Mamba backbone. The experimental design first locks a shared GLD baseline configuration, then compares clean baselines and NS-Mamba variants across horizons from 1 to

360 days. At the current stage, the manuscript is suitable as a research draft and implementation guide. It will become submission-ready only after the baseline table, NS-Mamba results, ablations, and discussion are completed with traceable quantitative evidence.

References

[1] Albert Gu and Tri Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. 2023.

[2] Zihan Wang, Fanheng Kong, Shi Feng, Ming Wang, Xiaocui Yang, Han Zhao, Daling Wang, and Yifei Zhang. Is Mamba Effective for Time Series Forecasting? arXiv:2403.11144, 2024.

[3] Aobo Liang, Xingguo Jiang, Yan Sun, Xiaohou Shi, and Ke Li. Bi-Mamba+: Bidirectional Mamba for Time Series Forecasting. arXiv:2404.15772, 2024.

[4] Xiuding Cai, Yaoyao Zhu, Xueyao Wang, and Yu Yao. MambaTS: Improved Selective State Space Models for Long-term Time Series Forecasting. arXiv:2405.16440, 2024.

[5] Md Atik Ahamed and Qiang Cheng. TimeMachine: A Time Series is Worth 4 Mambas for Long-term Forecasting. European Conference on Artificial Intelligence, 2024.

[6] Haoyu Ma, Yushu Chen, Wenlai Zhao, Jinzhe Yang, Yingsheng Ji, Xinghua Xu, Xiaozhu Liu, Hao Jing, Shengzhuo Liu, and Guangwen Yang. A Mamba Foundation Model for Time Series Forecasting. arXiv:2411.02941, 2024.

[7] Tae-Hyun Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift. International Conference on Learning Representations, 2022.

[8] Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. Non-stationary Transformers: Exploring the Stationarity in Time Series Forecasting. Advances in Neural Information Processing Systems, 2022.

[9] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. International Conference on Learning Representations, 2024.

[10] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. AAAI Conference on Artificial Intelligence, 2021.

[11] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. Advances in Neural Information Processing Systems, 2021.

[12] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The Efficient Transformer. International Conference on Learning Representations, 2020.